

university of groningen / faculty of science and engineering / mathematics and applied mathematics

Randomized approaches to approximating solutions of large kernel systems for Kernel Ridge Regression using the Nyström method

Bachelor's Project Mathematics

April 2026

Student: Y. Tao

First supervisor: Dr. M. Meier

Second supervisor: Dr. A. Smith

Abstract

Kernel ridge regression leads to a linear system involving the full kernel matrix K. This matrix has size $n \times n$, so forming and storing it becomes expensive when the number of data points is large. This project studies randomized Nyström approximations, where only $l \ll n$ columns of K are used to build a low-rank approximation. The aim is to compare how the choice of sampled columns and the number of sampled columns affect accuracy, runtime, memory use, conditioning, and solver behaviour.

Contents

1 Introduction **3**

2 Kernel Ridge Regression **4**
2.1 Introduction to the regression problem 4
2.2 Representer form 4
2.3 Kernel choices 6
2.4 Singular values and low-rank structure 6
2.5 Runtime and Accuracy experiments 7

3 Nyström Approximation **11**
3.1 General Nyström form 11
3.2 Column sampling 12
3.3 Randomized Nyström approximation 13
3.4 Uniform sampling 13

1 Introduction

Many regression problems start with data (x_i, y_i) for $i = 1, \ldots, n$, where $x_i \in \mathbb{R}^d$ are inputs and $y_i \in \mathbb{R}$ are outputs. We would like to use these data to create a model for the relationship between the x and y. One formulation of this task is kernel ridge regression, where the final computation is written using a kernel matrix. The details of this formulation are introduced in Chapter 2.

For large datasets, the main computational difficulty is the need to explicitly form the full matrix K, which costs $O(n^2)$ time and memory. This is the part that becomes infeasible first and takes the most time and resources. Solving the resulting system directly can also cost about $O(n^3)$ operations, so this is fine for small tests, but not realistic when n becomes large.

Therefore in this project we investigate the Nyström approximation of the kernel matrix: instead of forming all of K, we only form $l \ll n$ sampled columns and the corresponding $l \times l$ submatrix and make an approximation $\tilde{K} \approx K$. This is a randomized approach for kernel matrices [Drineas and Mahoney(2005), Gittens and Mahoney(2016)]. The point is to reduce the computational cost while still keeping enough information from the original kernel matrix.

Nyström needs a set of l columns to build $\tilde{K}$. The simplest choice is uniform random sampling, i.e. pick columns uniformly at random. Smarter strategies try to pick columns that are more informative so that fewer columns are needed for similar accuracy. One idea is to sample according to ridge leverage scores, which give higher probability to points that matter more for the kernel ridge regression solution [El Alaoui and Mahoney(2015)]. Another idea is to approximate these leverage type probabilities adaptively without ever constructing the whole K [Calandriello et al.(2017)].

This project is therefore about how to decide which columns to sample and how many columns l are needed for a good Nyström approximation in the KRR setting. I will compare different sampling strategies, starting from uniform random sampling as a baseline, and then comparing against smarter strategies motivated by the literature. The experiments will compare these strategies in terms of prediction accuracy and computational complexity, such as runtime and memory, as l changes.

The research questions are:

- What Nyström column sampling strategies give good performance for KRR in terms of prediction accuracy vs runtime/memory?
- How large does l need to be to get a good approximation, and how does this depend on the sampling strategy and λ?
- What do the accuracy and cost (runtime/memory) trade-offs look like when l increases, and can we recommend a reasonable approach?
- How do conditioning and solver convergence relate to λ and the Nyström approximation quality?

This thesis is organised as follows. Chapter 2 introduces kernel ridge regression and the kernel matrices used in the project. It also gives the first singular value experiments, which motivate why a low-rank approximation is a reasonable thing to try. Chapter 3 introduces the Nyström approximation, starting from the general positive semidefinite matrix formula and then moving to column sampling.

2 Kernel Ridge Regression

2.1 Introduction to the regression problem

In this section we introduce the regression problem that will be used throughout the project. Assume we have a dataset (x_i, y_i) for $i = 1, \ldots, n$ where $x_i \in \mathbb{R}^d$ are inputs and $y_i \in \mathbb{R}$ are outputs. We would like to use these data to create a model for the relationship between the x and y. In ordinary linear regression one searches over linear functions of the input variables. Kernel ridge regression instead searches over a reproducing kernel Hilbert space $\mathcal{H}$, which allows nonlinear functions to be represented through a kernel function.

Let

$$k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$$

be a positive definite kernel function. For the training data we define the kernel matrix

$$K \in \mathbb{R}^{n \times n}, \quad K_{ij} = k(x_i, x_j).$$

The entry K_{ij} measures the similarity between the data points x_i and x_j according to the chosen kernel function. The matrix K induced by k is symmetric positive semidefinite.

The kernel trick allows us to formulate this as

$$\min_{f \in \mathcal{H}} \|f(X) - y\|_2^2 + \lambda \|f\|_{\mathcal{H}}^2,$$

which is also

$$\min_{f \in \mathcal{H}} \sum_{i=1}^{n} (f(x_i) - y_i)^2 + \lambda \|f\|_{\mathcal{H}}^2.$$

Here $\lambda > 0$ is the regularization parameter. The first term measures the error on the training data while the second term penalizes functions with large RKHS norm. Here λ controls the trade off between fitting the data and overfitting.

2.2 Representer form

By the representer theorem, the minimizer has the form

$$f(x) = \sum_{j=1}^{n} \alpha_j k(x, x_j),$$

for some coefficient vector

$$\alpha = (\alpha_1, \ldots, \alpha_n)^T \in \mathbb{R}^n.$$

For the training points we have

$$f(x_i) = \sum_{j=1}^{n} \alpha_j k(x_i, x_j),$$

so in vector form

$$f(X) = K\alpha.$$

Therefore the data fitting term becomes

$$\sum_{i=1}^{n} (f(x_i) - y_i)^2 = \|K\alpha - y\|_2^2.$$

The RKHS norm term can also be written using the kernel matrix. First, by the reproducing property,

$$\langle k(\cdot, x_i), k(\cdot, x_j)\rangle_{\mathcal{H}} = k(x_i, x_j).$$

Since

$$f = \sum_{i=1}^{n} \alpha_i k(\cdot, x_i),$$

we have

$$\|f\|_{\mathcal{H}}^2 = \left\langle \sum_{i=1}^{n} \alpha_i k(\cdot, x_i), \sum_{j=1}^{n} \alpha_j k(\cdot, x_j) \right\rangle_{\mathcal{H}}.$$

Using bilinearity of the inner product,

$$\|f\|_{\mathcal{H}}^2 = \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j \langle k(\cdot, x_i), k(\cdot, x_j)\rangle_{\mathcal{H}}.$$

Using the reproducing property again,

$$\|f\|_{\mathcal{H}}^2 = \sum_{i=1}^{n}\sum_{j=1}^{n} \alpha_i \alpha_j k(x_i, x_j).$$

Since $K_{ij} = k(x_i, x_j)$ this becomes

$$\|f\|_{\mathcal{H}}^2 = \alpha^T K \alpha.$$

Substituting this into the objective gives

$$\min_{\alpha \in \mathbb{R}^n} \|K\alpha - y\|_2^2 + \lambda \alpha^T K \alpha.$$

Expand

$$\|K\alpha - y\|_2^2 = (K\alpha - y)^T (K\alpha - y).$$

Then differentiate

$$(K\alpha - y)^T (K\alpha - y) + \lambda \alpha^T K \alpha$$

with respect to α. Since K is symmetric this gives

$$2K(K\alpha - y) + 2\lambda K\alpha = 0.$$

This then reduces to

$$(K + \lambda I)\alpha = y.$$

Therefore the coefficient vector is

$$\alpha = (K + \lambda I)^{-1} y.$$

The prediction for a new point x is then

$$\hat{f}(x) = \sum_{j=1}^{n} \alpha_j k(x, x_j).$$

If X_{test} is a set of test points and $K_{\text{test,train}}$ is the matrix with entries

$$(K_{\text{test,train}})_{ij} = k(x_i^{\text{test}}, x_j^{\text{train}}),$$

then the test predictions are

$$\hat{y}_{\text{test}} = K_{\text{test,train}}\alpha.$$

The regularization term is important numerically. Since K can have very small eigenvalues or even zero eigenvalues, adding λI shifts all eigenvalues by λ so $K + \lambda I$ is positive definite. This makes the system uniquely solvable and improves the conditioning of the problem.

For large datasets, the main computational difficulty is that K is an $n \times n$ matrix. Forming the full kernel matrix requires $O(n^2)$ kernel evaluations and storing it requires $O(n^2)$ memory. Solving the system directly, for example by Cholesky decomposition, costs $O(n^3)$ operations. For small n this is fine but for large datasets this quickly becomes infeasible. Therefore in this project we investigate the Nyström approximation of the kernel matrix: instead of forming all of K, we only form $l \ll n$ sampled columns and the corresponding $l \times l$ submatrix, and then make a low rank approximation $\tilde{K} \approx K$.

2.3 Kernel choices

In the experiments we start with three common kernels. The linear kernel is

$$k(x, z) = x^T z.$$

The polynomial kernel used in the first tests is

$$k(x, z) = (x^T z + 1)^2.$$

The Gaussian radial basis function kernel (rbf) is

$$k(x, z) = \exp(-\gamma \|x - z\|_2^2).$$

The parameter γ controls how local the Gaussian kernel is. Larger γ makes points only strongly similar to very nearby points while smaller γ makes the similarities decay more slowly.

2.4 Singular values and low-rank structure

A kernel matrix is useful for Nyström approximation when its singular values decay. If many singular values are very small then much of the matrix can be approximated using a smaller rank. This is why in this section we look at singular values of K for real data. The point is to check whether a low-rank approximation is reasonable for real world data.

For this real data experiment we use the Superconductivity dataset from the UCI Machine Learning Repository. This is a regression dataset with 21263 samples and 81 numerical input features, and the output is the critical temperature. This fits the KRR setting since the target variable is continuous.

This dataset is also large enough to show the computational problem. If the full dataset is used, the kernel matrix has size $n \times n$ where $n = 21,263$, this means there are $n^2 \approx 10^8$ entries in K which corresponds to about 3.6 GB in memory, not to mention the construction computational cost.

This is before solving the linear system or computing any decomposition. So the full matrix is already expensive, and this is the reason that an approximation method such as Nyström is relevant.

We now look at the singular values of three kernels linear, polynomial, and Gaussian RBF. The singular values are computed using the full kernel matrix, so this is just a check of the low-rank structure. The point is to see whether the singular values decay in practice for real data.

For the singular value plots, the singular values are normalised by the largest singular value, so the plotted quantity is

$$\frac{\sigma_i(K)}{\sigma_1(K)}.$$

This makes the three kernels easier to compare since all three curves start at 1. The plots are also stopped once the singular values reach around 10^{-16}, since after that the remaining tail is python or machine precision.

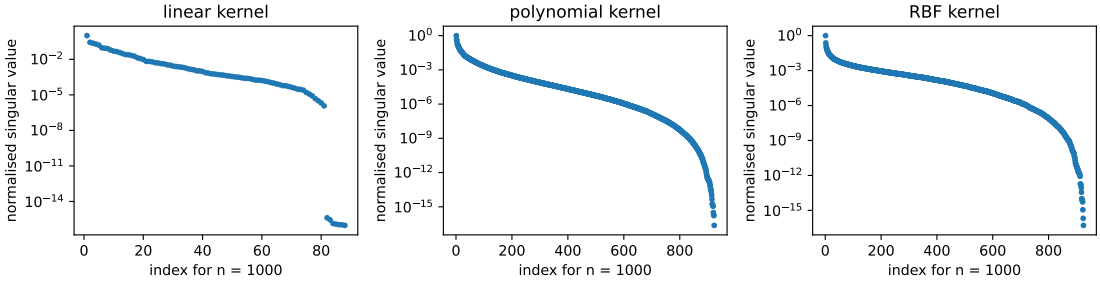


Figure 2.1: Normalised singular value decay for the three kernel matrices: (a) linear, (b) polynomial and (c) RBF.

Figure 2.1 compares the three kernels used in this project. Figure 2.1(a) shows the linear kernel. Since $K = XX^T$ for a linear kernel and the data have 81 features, the rank is at most 81, so the singular values drop to zero after that point. Figure 2.1(b) shows the polynomial kernel. This kernel is still based on inner products, but the square introduces nonlinear factors, so its spectrum does not have the same clean cut-off as the linear kernel. Figure 2.1(c) shows the RBF kernel. Here the decay depends more on the geometry of the data and the value of γ, so this is the most interesting case out of all three kernels.

2.5 Runtime and Accuracy experiments

We now check the computational cost in this section. Constructing a full kernel matrix means evaluating pairwise similarities between observations so the expected cost is quadratic in n. Solving the full KRR has cubic asymptotic cost if we use Cholesky or some other direct solver. However in the tested range the construction of K can still take longer than the solve. This does not contradict the asymptotic costs. It just means the $O(n^2)$ construction curve can start much higher since every kernel entry has to be evaluated and stored while the solve is done by highly optimized linear algebra methods. In practice construction is almost always more expensive than the solve.

Figure 2.2 shows the construction time for the linear, polynomial and RBF kernels. The general behaviour follows the expected quadratic growth, but the constants are not the same. RBF kernel is slower since it is a more complex function than the other two.

Figure 2.3 compares the time to construct the RBF kernel matrix with the time to solve $(K + \lambda I)\alpha = y$. The construction time is larger for the increasing values of n which agrees with the theoretical expectation we have discussed above. This is one of the main reasons why column sampling is important: if only l columns are sampled, then we can build $K_{:,I}$ and $K_{I,I}$ instead of all of K which we will see in Chapter 3.

For the regression part, we fix a train/test split and first vary the regularization parameter λ. The metric is MSE (mean squared error).

Figure 2.4 shows we also study the choice of λ. Both very small or very large λ give less stable fits, this is as expected since small lambda underregularizes which causes overfitting, and large lambda overregularizes which causes underfitting. The best value in this small experiment is in the middle of the tested range at roughly $\lambda = 0.1$.

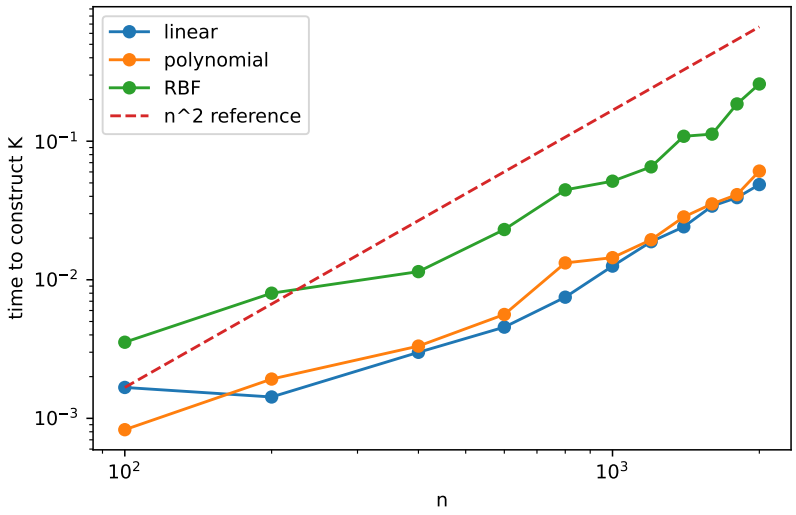


Figure 2.2: Time to construct the full kernel matrix for different values of n.

Finally, we fix the test set and use increasingly larger subsets of the training set. This checks both the prediction error and the runtime as the number of training observations grows.

Figures 2.5 and 2.6 show the basic trade off. More training observations can improve the test error but the runtime increases at the same time. This is the practical situation where Nyström becomes very useful: ideally we want to keep majority of the predictive information while avoiding the computational cost of dealing with the full kernel matrix.

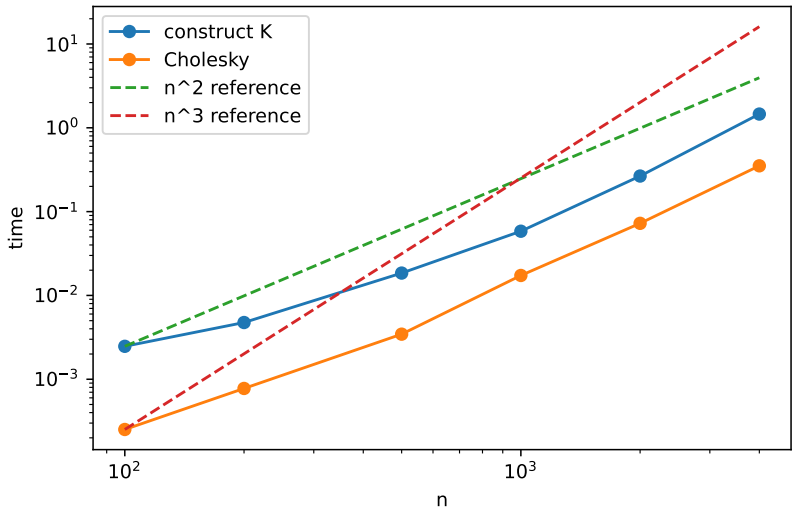


Figure 2.3: Comparison of the time to construct K and the time to solve the full KRR system.

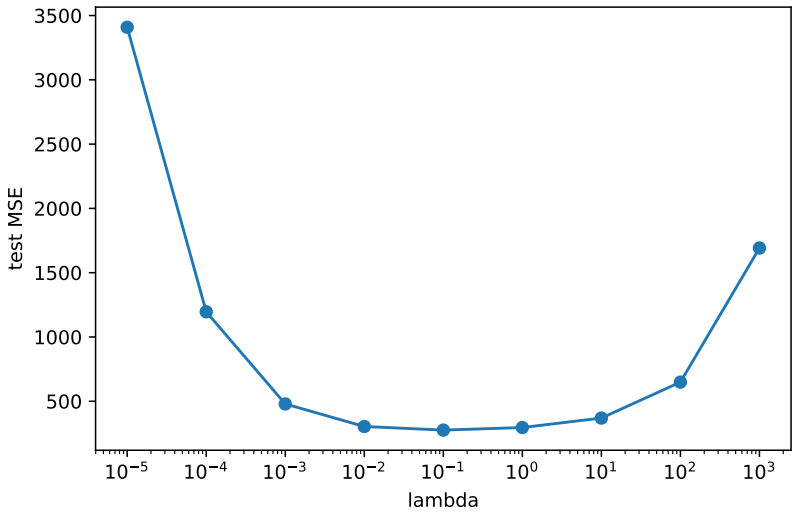


Figure 2.4: Test MSE for full KRR as λ changes.

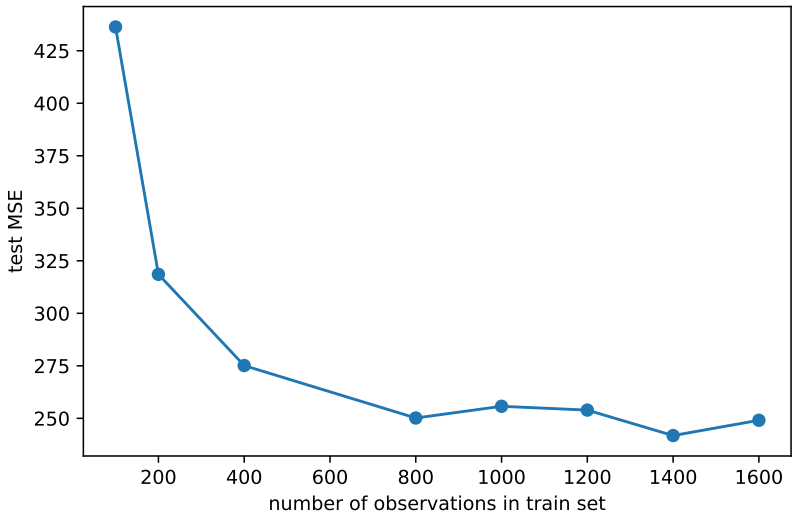


Figure 2.5: Test MSE as the number of training observations increases.

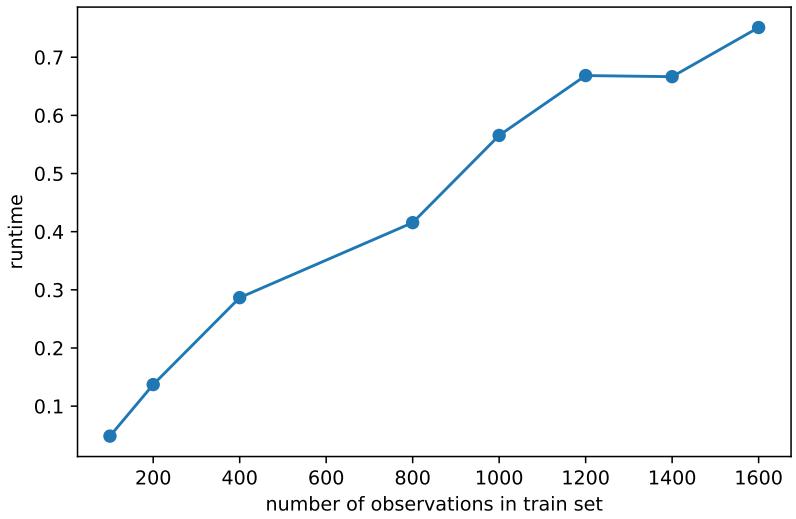


Figure 2.6: Runtime as the number of training observations increases.

3 Nyström Approximation

The Nyström method is a way to approximate a large positive semidefinite matrix by using only a smaller part of it. In this project the positive semidefinite matrix is the kernel matrix K, but it is useful to first describe the construction for a general positive semidefinite matrix A. The aim is to build a low rank approximation $\tilde{A}$ without using all of A in a full direct factorization.

3.1 General Nyström form

Following the general formulation in randomized numerical linear algebra, let

$$A \in \mathbb{R}^{n\times n}, \quad A \succeq 0,$$

and let

$$X \in \mathbb{R}^{n\times l}$$

be a test matrix, the Nyström approximation of A with respect to X is

$$A\langle X\rangle = (AX)(X^TAX)^\dagger(AX)^T.$$

Here $(X^TAX)^\dagger$ denotes the Moore–Penrose pseudoinverse. The matrix AX contains the information we collect from A. The smaller matrix

$$X^TAX \in \mathbb{R}^{l\times l}$$

describes how the sampled directions interact with each other. This is the same general positive semidefinite sketching model used by Gittens and Mahoney, where one forms $C = AS$, $W = S^TAS$, and then uses $CW^\dagger C^T$ as the approximation [Gittens and Mahoney(2016)].

This formula shows why the approximation has rank at most l since the formula is a product involving the $n \times l$ matrix AX, so

$$\operatorname{rank}(A\langle X\rangle) \leq l.$$

Therefore, when $l \ll n$, the approximation is much cheaper to store and compute with than the full matrix A.

There is also a useful geometric representation of this formula. Since A is positive semidefinite we can use $A^{1/2}$. Then

$$A\langle X\rangle = A^{1/2}P_{A^{1/2}X}A^{1/2},$$

where $P_{A^{1/2}X}$ is the orthogonal projector onto the column space of $A^{1/2}X$. This has error as

$$A - A\langle X\rangle = A^{1/2}(I - P_{A^{1/2}X})A^{1/2}$$

which is the Schur complement viewpoint used by Martinsson and Tropp [Martinsson and Tropp(2020)]. This means that the Nyström approximation keeps the part of A which is seen through the subspace generated by X.

Theorem 3.1. *Let $A \in \mathbb{R}^{n\times n}$ be positive semidefinite and let $X \in \mathbb{R}^{n\times l}$. Then*

$$A - A\langle X\rangle = A^{1/2}(I - P_{A^{1/2}X})A^{1/2}.$$

In particular,

$$A - A\langle X\rangle \succeq 0,$$

so

$$0 \preceq A\langle X\rangle \preceq A.$$

Proof. Let

$$Z = A^{1/2}X.$$

The orthogonal projector onto the column space of Z is

$$P_Z = Z(Z^TZ)^\dagger Z^T.$$

Since $Z = A^{1/2}X$, we have

$$Z^TZ = X^TAX.$$

Therefore

$$A^{1/2}P_ZA^{1/2} = A^{1/2}A^{1/2}X(X^TAX)^\dagger X^TA^{1/2}A^{1/2}.$$

This gives

$$A^{1/2}P_ZA^{1/2} = (AX)(X^TAX)^\dagger(AX)^T = A\langle X\rangle.$$

Hence

$$A - A\langle X\rangle = A^{1/2}IA^{1/2} - A^{1/2}P_ZA^{1/2} = A^{1/2}(I - P_Z)A^{1/2}.$$

Since $I - P_Z$ is an orthogonal projector, it is positive semidefinite. Thus

$$A - A\langle X\rangle \succeq 0.$$

This also gives

$$0 \preceq A\langle X\rangle \preceq A.$$

□

This is useful because it says that the Nyström approximation does not create a matrix larger than A in the positive semidefinite order. It approximates A from below. In the KRR setting this matters because replacing K by $\tilde{K}$ changes the regularized system from

$$(K + \lambda I)\alpha = y$$

to

$$(\tilde{K} + \lambda I)\tilde{\alpha} = y.$$

Since $\tilde{K} \preceq K$, the approximation changes both the estimator and the conditioning of the linear system. This is one reason why the number of sampled columns l, the sampling strategy and the regularization parameter λ have to be studied together.

3.2 Column sampling

We now look at column sampling. Let

$$I = \{i_1, \dots, i_l\} \subset \{1, \dots, n\}$$

be the selected column indices. Define the sampling matrix

$$S = \begin{bmatrix} e_{i_1} & e_{i_2} & \cdots & e_{i_l} \end{bmatrix} \in \mathbb{R}^{n\times l},$$

where e_i is the i-th standard basis vector. Taking $X = S$ in the general Nyström formula gives

$$\tilde{K} = K\langle S\rangle = (KS)(S^TKS)^\dagger(KS)^T.$$

Now define

$$C = KS = K_{:,I} \in \mathbb{R}^{n\times l}$$

and

$$W = S^TKS = K_{I,I} \in \mathbb{R}^{l\times l}.$$

Then the Nyström approximation becomes

$$\tilde{K} = CW^{\dagger}C^{T}.$$

Equivalently,

$$\tilde{K} = K_{:,I}(K_{I,I})^{\dagger}K_{I,:}.$$

This is the form used in this project. Instead of forming or storing all of K, the approximation is built from the sampled columns $K_{:,I}$ and the smaller intersection matrix $K_{I,I}$. If W is invertible then the formula simplifies to

$$\tilde{K} = CW^{-1}C^{T}.$$

However in practice $W^{\dagger}$ is safer to write since W can be singular or nearly singular.

The computational meaning is obviously seen. The full kernel matrix has n^2 entries. The column sampling form only needs the $n \times l$ matrix C and the $l \times l$ matrix W. So the storage goes from order n^2 to order $nl + l^2$. This becomes significantly useful when $l \ll n$.

3.3 Randomized Nyström approximation

The randomized Nyström approximation draws a random matrix

$$\Omega \in \mathbb{R}^{n \times l}$$

and form

$$Y = A\Omega.$$

Then the Nyström approximation is

$$\hat{A} = Y(\Omega^{T}Y)^{\dagger}Y^{T}.$$

This is the randomized Nyström approximation described by Martinsson and Tropp [Martinsson and Tropp(20
The idea is that the random matrix Ω collects information about the important directions of A. If the column space of $A\Omega$ captures the dominant eigenspace of A well then the approximation error should be small.

For kernel matrices the usual Nyström method samples columns of K. This corresponds to taking $\Omega = S$, where S is a column selection matrix. Then

$$Y = KS = K_{:,I},$$

and the formula becomes

$$\tilde{K} = K_{:,I}(K_{I,I})^{\dagger}K_{I,:}.$$

Thus in this project randomness enters through the random choice of the index set I. Different choices of the distribution for I give different Nyström methods.

3.4 Uniform sampling

The baseline method is uniform sampling. This means choosing l column indices from $\{1, \ldots, n\}$ uniformly at random. It is easy to implement and cheap because no importance scores have to be computed. Its weakness is that it treats all columns equally. If some columns are much more informative than others, uniform sampling may need a larger l to reach the same accuracy as a smarter strategy.

This issue is discussed by Gittens and Mahoney. In some datasets the leverage scores are fairly uniform so uniform sampling can be reasonable. In other datasets the leverage scores are highly nonuniform and uniform sampling can be much worse [Gittens and Mahoney(2016)].

This is why uniform sampling is a natural baseline method but not necessarily the best method.

In the first experiments we test uniform sampling. For each l, the approximation quality can be measured directly by a relative forward error, for example

$$\frac{\|K - \tilde{K}\|_F}{\|K\|_F}.$$

It is also important to measure the effect on prediction. So the approximation is also judged by what happens when $\tilde{K}$ is used in place of K in the regularized system, this is done by solving the system with $\tilde{K}$ and then measuring the test error of the result.

Bibliography

[Calandriello et al.(2017)] Calandriello, D., Lazaric, A., and Valko, M. (2017). Distributed adaptive sampling for kernel matrix approximation. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*.

[Drineas and Mahoney(2005)] Drineas, P. and Mahoney, M. W. (2005). On the Nystrom method for approximating a Gram matrix for improved kernel-based learning. *Journal of Machine Learning Research*, 6, 2153–2175.

[El Alaoui and Mahoney(2015)] El Alaoui, A. and Mahoney, M. W. (2015). Fast randomized kernel methods with statistical guarantees. In *Advances in Neural Information Processing Systems*.

[Gittens and Mahoney(2016)] Gittens, A. and Mahoney, M. W. (2016). Revisiting the Nystrom method for improved large-scale machine learning. *Journal of Machine Learning Research*, 17, 1–65.

[Martinsson and Tropp(2020)] Martinsson, P.-G. and Tropp, J. A. (2020). Randomized numerical linear algebra: foundations and algorithms. *Acta Numerica*.

[Zhu(2025)] Zhu, R. (2025). Reproducing kernel Hilbert space. *Statistical Machine Learning with R*.

[Hamidieh(2018)] Hamidieh, K. (2018). Superconductivty Data. UCI Machine Learning Repository. DOI: 10.24432/C53P47.